

# Trevolto

Automated crypto trading — the desktop engine, in the browser. Generates its own signals, auto-executes trades and manages risk 24/7.

Full-stack automated crypto trading platform · FastAPI + ccxt · v2.1 · August 1, 2026

## 1 What is Trevolto?

Trevolto is a multi-user, non-custodial crypto trading platform. Each customer connects their own exchange with a trade-only API key (withdrawals disabled), and the bot runs a fully-automated strategy 24/7 on their account — generating its own buy/sell signals, executing entries and exits, and managing risk. One FastAPI server hosts many users; funds never leave the user's exchange.

Browser dashboard ↗  
 TradingView webhook ↔ FastAPI → per-user TraderSession → ccxt → exchange  
 Built-in strategy ↙ (guardrails → sizing → order → SL/TP bracket)



### Fully-automated strategy

EMA 9/21 crossover + 100-EMA trend filter, confirmation candles, swing-based stops, partial take-profit + trailing exit. ON by default; fully tunable.



### Profit Simulator

Replays the exact live strategy on real market history — win rate, drawdown, equity curve, every trade.



### Live signal-strength meter UPDATED

A 0–100% gauge shows the next trade forming; at 85% it fires one “get ready” alert per build-up — rate-limited so it can't spam you (see \$7).



### High-impact news filter

One-tap switch to hold new entries 1h before & after major USD events (FOMC, CPI, jobs). Open trades stay managed.



### Weekend pause

Optionally hold new entries from Saturday until Sunday evening, then auto-resume for the market re-open.



### Telegram — two-way control

Entry/close alerts + daily PnL recap, and run the bot from chat: /status /pnl /positions /pause /resume /closeall.



### In-app alerts inbox NEW

Bell panel shows every signal & trade alert, with a one-tap **Clear all** button to empty the inbox (persists across reloads).



### VIP Custom Strategy

Advanced users unlock every parameter — a one-time \$497 for lifetime access, granted via an invoice flow by the admin.



### Security

Fernet-encrypted trade-only keys, 2FA (TOTP), login history, sign-out-all, withdrawal-key warning, IP-whitelist helper.



### Scale-in (pyramiding)

Adds a smaller same-direction trade once 40% to TP and moves the original stop to break-even.



### Diagnostics

Public /api/health + authenticated /api/selftest (DB, exchange ping, feed, strategy loop, licence) with a one-click Settings button.

Binance

Binance.US

Bybit

OKX

KuCoin

Bitget

Kraken

Coinbase

— spot & futures

## 2 Project structure & stack

| Path           | What it is                                                                    |
|----------------|-------------------------------------------------------------------------------|
| server/main.py | FastAPI app — REST + WebSocket routes, auth, admin, webhooks, health/selftest |

|                       |                                                                                                              |
|-----------------------|--------------------------------------------------------------------------------------------------------------|
| server/session.py     | Per-user TraderSession — strategy loop, order flow, live state, weekend/news guards, signal-alert rate-limit |
| server/store.py       | SQLite (WAL) data layer — users, keys, trades, licences, audit, one-click export/import                      |
| server/security.py    | Password hashing, JWT, Fernet key encryption, TOTP 2FA                                                       |
| server/engine/        | Trading core — exchange.py (ccxt), strategy.py, guardrails.py, backtest.py                                   |
| web/                  | Dashboard front-end — index.html, app.js, admin.js, style.css (vanilla JS + TradingView Lightweight Charts)  |
| Dockerfile · fly.toml | Container + Fly.io deploy config                                                                             |

**Stack** Python 3.11 · FastAPI · ccxt · SQLite · Fernet · PyJWT · WebSockets · vanilla JS.

### 3 Run locally

```
# from the repo root
cd app.trevolto
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export TREVOLTO_SECRET_KEY="$(python -c 'import secrets;print(secrets.token_urlsafe(48))')"
uvicorn server.main:app --host 0.0.0.0 --port 8000
# open http://localhost:8000 (first registered user becomes admin)
```

| Environment variable | Purpose                                                                                              |
|----------------------|------------------------------------------------------------------------------------------------------|
| TREVOLTO_SECRET_KEY  | <b>Required.</b> Long random string — encrypts API keys & signs JWTs. Protect with a secret manager. |
| TREVOLTO_ADMIN_EMAIL | Email that becomes admin on registration.                                                            |
| TREVOLTO_PUBLIC_URL  | Your public base URL (e.g. <code>https://yourdomain.com</code> ) — used in reset links & webhooks.   |
| TREVOLTO_DATA_DIR    | Where the SQLite DB lives — point at a persistent path so data survives restarts.                    |
| TREVOLTO_TRIAL_DAYS  | Free-trial length (default 10).                                                                      |

## 4 Move to any server — one-click install

Trevolto is a standard FastAPI service, so it runs on any VPS with SSH + Python — Hostinger VPS, GoDaddy VPS, DigitalOcean, Contabo and the like. A bundled installer does the entire setup for you.

```
# upload (or git clone) the app folder to the server, then:
cd app.trevolto
sudo bash install.sh
```

It asks for your domain and admin email, then automatically:

- installs Python, Nginx & all dependencies
- creates a virtualenv and installs `requirements.txt`
- generates & saves a strong `TREVOLTO_SECRET_KEY` into `/etc/trevolto.env`
- installs a systemd service — auto-starts on boot, auto-restarts on crash
- configures an Nginx reverse proxy with WebSocket support
- issues a free auto-renewing HTTPS certificate (Let's Encrypt)

Then open `https://yourdomain.com` — the first account you register becomes the admin. Manage with `systemctl status|restart|stop trevolto`; to update, re-upload files (or `git pull`) then `systemctl restart trevolto`. Re-running `install.sh` is safe — it keeps your existing secret key & database.

**No VPS? Hostinger / cPanel shared hosting.** On cPanel → “Setup Python App”: set the app root to the uploaded folder, startup file `passenger_wsgi.py`, Python 3.11, install `requirements.txt`, and add the env vars. Passenger doesn't proxy WebSockets, so the dashboard auto-falls back to 3-second polling (still fully functional). Full step-by-step in the Appendix.

### Quick alternative — Fly.io

```
cd app.trevolto
fly deploy
fly secrets set TREVOLTO_SECRET_KEY=... TREVOLTO_ADMIN_EMAIL=...
```

GitHub Actions also auto-deploys on push to main. The marketing site (`site.trevolto/`) deploys separately to Hostinger from the `site.trevolto` branch.

## 5 Admin panel (/admin)

- **Every customer:** status, keys, last-seen, PnL — suspend/activate, grant/revoke licences (N days or lifetime).
- **Custom-strategy (VIP) requests:** mark an invoice paid to unlock a customer's custom strategy (\$497).
- **Global managed strategy** (pushed to all managed users), broadcasts, social-proof, audit log.
- **One-click database backup & restore** — export the whole DB (customers, encrypted keys, trades, licences) as a file and import it on another server. Keep the same `TREVOLTO_SECRET_KEY` so encrypted keys still decrypt.

First registered user is admin; or set `TREVOLTO_ADMIN_EMAIL`.

## 6 VIP Custom Strategy — invoice flow

1. Admin grants a lifetime licence → the customer sees the ★ Unlock VIP offer.
2. Customer taps **Request access** → shows in the admin requests panel.
3. Admin emails an invoice for \$497; on payment clicks **Mark invoice paid**.
4. Custom Strategy unlocks instantly; the customer's top bar shows the ★ VIP Unlocked badge.

## 7 Signal alerts & Telegram control NEW

The dashboard's signal-strength meter fires an early “get ready” alert as a trade sets up, and the whole bot can be driven from Telegram. Both are covered in-app under ⚡ **How it works** → **step 7**.

**No more alert spam.** The 85% “get ready” alert now fires **once per genuine build-up** and **at most once every 30 minutes per symbol**. It only re-arms after strength drops back below 70%, so a choppy market that wobbles 85 → dip → 85 sends a single alert instead of flooding your inbox. In-app, the bell panel's **Clear all** button empties the inbox anytime.

### Create your Telegram bot (≈3 min):

1. Open **@BotFather** in Telegram → send `/newbot` → name it → copy the **bot token**.
2. Open **@userinfobot** → tap Start → copy your numeric **Chat ID**.
3. Tap **Start** once on your new bot so it can message you.
4. In the app: **Settings** → **Telegram notifications**, paste the token + Chat ID, tap  **Send test**.

### Then control it from any chat:

|                                            |                                             |
|--------------------------------------------|---------------------------------------------|
| <code>/status</code>                       | Connection, strategy, balance & open trades |
| <code>/pnl</code>                          | Realized PnL — 24h + all-time               |
| <code>/positions</code>                    | List every open position                    |
| <code>/pause</code> · <code>/resume</code> | Stop / start opening new trades             |
| <code>/closeall</code>                     | Close all open positions now                |
| <code>/help</code>                         | Show the full command list                  |



Only your own linked chat ID can send commands — nobody else can control your bot, even if they learn its name.

## Appendix · cPanel / shared hosting — step by step

No VPS or SSH? Any cPanel host with “Setup Python App” (Hostinger shared, GoDaddy cPanel, Namecheap...) runs Trevolto through Passenger. WebSockets aren't proxied, so the dashboard auto-uses 3-second polling — everything still works.

**1 • Upload the app.** cPanel → File Manager → your home folder (keep it outside `public_html`). Upload the zip and Extract it — e.g. `/home/USER/trevolto`.

**2 • Create the Python app.** cPanel → Setup Python App → Create Application:

| Field                    | Value                           |
|--------------------------|---------------------------------|
| Python version           | 3.11 (or the highest available) |
| Application root         | <code>trevolto</code>           |
| Application URL          | your domain or subdomain        |
| Application startup file | <code>passenger_wsgi.py</code>  |
| Application Entry point  | <code>application</code>        |

**3 • Install dependencies.** On the app's page, add `requirements.txt` and click Run Pip Install — or run `pip install -r requirements.txt` in cPanel Terminal after the shown source `.../activate`.

**4 • Set environment variables.** Same page, under Environment variables, add:

```
TREVOLTO_SECRET_KEY = <a long random string, 40+ chars>
TREVOLTO_PUBLIC_URL  = https://yourdomain.com
TREVOLTO_DATA_DIR    = /home/USER/trevolto/data
TREVOLTO_ADMIN_EMAIL = you@email.com
```

Need a secret? Run `python -c "import secrets;print(secrets.token_urlsafe(48))"` on any PC.

**5 • Restart & open.** Click Restart on the app page, then open your domain — the first account you register becomes the admin. Updating later: re-upload changed files, then Restart.

⚠ **Trading risk & custody.** Users' exchange keys are stored encrypted server-side and the server trades on their behalf — the operator is a custodian. Protect `TREVOLTO_SECRET_KEY` with a real KMS, require trade-only keys (withdrawals off), and keep users in paper/safe mode until validated. Provided as-is, no warranty. Trading involves risk and can lose money.